PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

Documento de Apoyo No. 9.1 – Espacio de trabajo ROS 2 robots_ws: clonación y construcción paso a paso.

Duración Estimada: 1 hora (trabajo independiente previo al bloque 2 del Día 2).

Sesión: Día 2 – 03/11/2026, antes de la configuración del RoboClaw (DA-10.1).

Relación con la Guía de Aprendizaje: Actividad 3.3.1 – carpeta «9. ESPACIO DE TRABAJO ROS 2».

Diseñado por: Julián René Cháux Cedeño.

Nombre del participante: _____ **Fecha:** _____

1. OBJETIVOS DE APRENDIZAJE

- Comprender qué es un espacio de trabajo de ROS 2 y en qué equipos se usa durante el curso.
- Clonar el espacio de trabajo común `~/robots_ws` desde GitHub y cargarlo en cada terminal.
- Agregar paquetes uno a uno con `git sparse-checkout` y recibir las correcciones con `git pull`.
- Saber qué espacio de trabajo está activo en una terminal (equivalente de `roscd`).
- Agregar los primeros paquetes del robot y probar `check_roboclaw` con el RoboClaw virtual.

2. FUNDAMENTOS

2.1 Espacio de trabajo, underlay y overlay

Un **espacio de trabajo** (*workspace*) es una carpeta con una subcarpeta `src` que contiene paquetes. `colcon` compila cada paquete y genera `build/`, `install/` y `log/`. Al ejecutar `source install/setup.bash`, los paquetes del espacio de trabajo se **superponen** (*overlay*) a la instalación de ROS en `/opt/ros/humble` (*underlay*): si un paquete existe en ambos, se usa el del espacio de trabajo [3]. `rosdep` lee el `package.xml` de cada paquete e instala con `apt` las dependencias que falten [5].

2.2 Un solo espacio de trabajo, en GitHub: ~/robots_ws

Todo el curso usa **un único espacio de trabajo, `~/robots_ws`**. En él conviven los paquetes del robot móvil (`mr_*`) y, si el equipo también trabaja con el cobot, los de la celda (`jc_*`): así la integración robot móvil – cobot (DA-21.1) compila en el mismo lugar.

Ese espacio de trabajo es el repositorio **https://github.com/jchauxdev/robots_ws** [8]. Se clona una vez, **vacío**, con la opción `--sparse` de git [9], y cada documento de apoyo, en su **Paso 0**, descarga solo los paquetes que va a usar con `git sparse-checkout add`. Si el instructor corrige un paquete, lo sube al repositorio y cada equipo lo recibe con `git pull`: no hay copias que mantener al día.

2.3 ¿En el PC o en el robot? En los dos

Tabla 1. Dónde se usa el espacio de trabajo robots_ws.

Equipo	Para qué se usa robots_ws	Cuándo se clona
PC del participante (Ubuntu 22.04)	Robot virtual, RViz2, SLAM, Nav2, App web y edición del código	En este documento
Computador del robot (Jetson AGX Orin, Orin Nano o Raspberry Pi 4, Ubuntu 22.04)	Drivers del robot (RoboClaw, lidar, cámaras) y, en la AGX, también SLAM y Nav2	Al final de su documento de instalación (carpeta 11)
Jetson Nano (Ubuntu 18.04)	Drivers del robot, compilados dentro del contenedor Docker	En el DA-11.2

Desde la carpeta 11, cada Paso 0 se hace en el PC y en el computador del robot, para que los dos tengan los mismos paquetes. La Tabla 2 indica en qué documento se agrega cada uno:

Tabla 2. Paquetes del curso y documento en el que se agregan a robots_ws.

Paquete o carpeta	Contenido	Se agrega en
src/mr_base	Driver ROS 2 del RoboClaw, odometría, herramientas `check_roboclaw` y `mover`; regla udev	DA-9.1
src/mr_sim	Robot virtual: RoboClaw emulado por un puerto serie virtual y lidar simulado	DA-9.1
src/mr_description	Modelo URDF/Xacro del robot con las mallas de SolidWorks	DA-12.1
src/mr_bringup	Lanzadores del robot real y virtual, SLAM, Nav2 y RViz2; parámetros	DA-12.1
systemd/	Servicio de arranque automático de los drivers	DA-13.3
src/mr_perception	Cámara USB con OpenCV	DA-15.1
src/mr_interfaces	Mensajes, servicios y acciones propios (órdenes, estaciones, puntos)	DA-18.1
src/mr_navigation	Puntos con nombre y orden «ir al punto X»	DA-18.1
src/mr_logistics	Gestor de pedidos y App web	DA-19.1
src/mr_cobot_bridge	Enlace con el cobot (con `jc_interfaces`)	DA-21.1
docker/	Contenedor ROS 2 de la Jetson Nano	DA-11.2

3. MATERIALES

- PC con Ubuntu 22.04 (nativo) y conexión a Internet.
- Usuario con permisos `sudo`.
- Para la prueba con el robot (DA-10.1, punto 5): RoboClaw configurado y cable USB.

4. PROCEDIMIENTO (EN EL PC)

Paso 1. Verificar (o instalar) ROS 2 Humble

Terminal – PC

```
# debe mostrar: Ubuntu 22.04.x LTS
lsb_release -ds

source /opt/ros/humble/setup.bash

# debe mostrar: humble
printenv ROS_DISTRO
```

Si `/opt/ros/humble` no existe, instale ROS 2 Humble con el procedimiento oficial [2]:

Terminal – PC

```
# verificar UTF-8
locale

sudo apt update && sudo apt install -y locales

sudo locale-gen en_US en_US.UTF-8

sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

export LANG=en_US.UTF-8

sudo apt install -y software-properties-common

sudo add-apt-repository universe

sudo apt update && sudo apt install -y curl

export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest | grep -F "tag_name" | awk -F'"' '{print $4}')

curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VERSION}.$(. /etc/os-release && echo ${UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"

sudo dpkg -i /tmp/ros2-apt-source.deb

# obligatorio antes de instalar ROS 2 en 22.04
sudo apt update && sudo apt upgrade -y

sudo apt install -y ros-humble-desktop ros-dev-tools

source /opt/ros/humble/setup.bash && printenv ROS_DISTRO
```

Paso 2. Herramientas: git y rosdep

Terminal – PC (en el robot se repite en la carpeta 11)

```
sudo apt install -y git tree python3-serial

# solo la primera vez en el equipo (si ya se hizo, muestra un aviso)
sudo rosdep init

rosdep update
```

Paso 3. Clonar el espacio de trabajo ~/robots_ws (vacío)

Si el equipo ya trabajó la celda del cobot, `~/robots_ws` ya está clonado: compruébelo y **no lo vuelva a clonar**.

Terminal – PC (en el robot se repite en la carpeta 11)

```
ls ~/robots_ws 2>/dev/null || echo "robots_ws no existe todavía"
```

```
git clone --filter=blob:none --sparse https://github.com/jchauxdev/robots_ws.git ~/robots_ws
```

```
# README.md (todavía sin paquetes)
ls ~/robots_ws
```

```
mkdir -p ~/robots_ws/src
```

```
cd ~/robots_ws
```

```
colcon build
```

Resultado esperado:

`Summary: 0 packages finished` y las carpetas `build/`, `install/` y `log/`. Es lo esperado: los paquetes se agregan en el Paso 7 y en los documentos siguientes. Si el espacio de trabajo ya existía, `colcon` compila los paquetes `jc_*` que tenga.

Precaución:

No copie paquetes a mano dentro de `~/robots_ws/src`: agréguelos con `git sparse-checkout add` para que `git pull` los mantenga al día. Sus propios paquetes sí se crean directamente en `src/`.

Paso 4. Cargar el entorno en cada terminal

Abra el archivo `~/.bashrc` con un editor de texto:

Terminal – PC (en el robot se repite en la carpeta 11)

```
# en el escritorio (si no está instalado: sudo apt install gedit)
gedit ~/.bashrc
```

```
# por SSH o sin escritorio
nano ~/.bashrc
```

Pegue al final del archivo las líneas del recuadro siguiente (solo las que no tenga ya; si hizo la transferencia del cobot, las dos primeras ya están), guarde (Ctrl + S en gedit; Ctrl + O, Enter y Ctrl + X en nano) y cierre el editor. Estas líneas no se escriben en la terminal: se cargan solas en cada terminal nueva.

Contenido para ~/.bashrc – se pega en el editor, no en la terminal

```
source /opt/ros/humble/setup.bash
[ -f ~/robots_ws/install/setup.bash ] && source ~/robots_ws/install/setup.bash
source /usr/share/colcon_cd/function/colcon_cd.sh
export _colcon_cd_root=~/robots_ws
```

Terminal – PC (en el robot se repite en la carpeta 11)

```
source ~/.bashrc
```

La condición `[ -f … ]` carga el espacio de trabajo solo si ya está compilado. Las dos últimas líneas activan `colcon_cd`, que se usa en el Paso 5. Las variables de red de ROS 2 (`ROS_DOMAIN_ID`) se agregan en el DA-13.2.

Paso 5. ¿Qué espacio de trabajo está activo en esta terminal?

En ROS 1, `roscd` llevaba a la carpeta del espacio de trabajo activo. En ROS 2 el espacio de trabajo activo es el que se cargó con `source …/install/setup.bash` en esa terminal (directamente o desde `~/.bashrc`), y se consulta con estos comandos [3], [4]:

Tabla 3. Equivalentes en ROS 2 para saber qué espacio de trabajo está activo.

Pregunta	Comando	Qué muestra
¿Qué espacios de trabajo están cargados?	`printenv COLCON_PREFIX_PATH`	La carpeta `install` de cada espacio de trabajo cargado, por ejemplo `/home/usuario/robots_ws/install`. Si no muestra nada, solo está cargado ROS 2 (`/opt/ros/humble`).
¿Todas las rutas de paquetes, en orden?	`printenv AMENT_PREFIX_PATH | tr ':' '\n'`	Primero los paquetes del espacio de trabajo (*overlay*) y al final `/opt/ros/humble` (*underlay*).
¿De dónde sale un paquete?	`ros2 pkg prefix mr_base`	La carpeta de instalación del paquete: dice si viene de `~/robots_ws` o de `/opt/ros/humble`.
Ir a la carpeta de un paquete (como `roscd <paquete>`)	`colcon_cd mr_base`	Cambia a `~/robots_ws/src/mr_base` (requiere la configuración del paso anterior).
Ir a la raíz del espacio de trabajo (como `roscd`)	`colcon_cd`	Cambia a `~/robots_ws`.

Terminal – PC

```
# espacios de trabajo cargados en esta terminal
printenv COLCON_PREFIX_PATH

# ir a la raíz del espacio de trabajo (como roscd)
colcon_cd && pwd
```

Resultado esperado:

`/home/<usuario>/robots_ws/install` y `/home/<usuario>/robots_ws`. Los comandos que consultan un paquete (`ros2 pkg prefix mr_base` y `colcon_cd mr_base`) todavía dan error porque `robots_ws` aún no tiene paquetes: se prueban en el Paso 7, después de agregar `mr_base`.

Precaución:

Cargue un solo espacio de trabajo propio por terminal. Si `printenv COLCON_PREFIX_PATH` muestra varias carpetas (por ejemplo un espacio de trabajo viejo), `ros2` puede usar una copia distinta del paquete que usted está editando: revise las líneas `source` de `~/.bashrc` y abra una terminal nueva.

Paso 6. Cómo se agrega un paquete (Paso 0 de cada documento)

Todos los documentos siguientes empiezan con un **Paso 0** de esta forma: se agrega la carpeta del paquete desde GitHub, se instalan sus dependencias y se compila.

Terminal – el equipo que indique el documento

```
cd ~/robots_ws
```

```
git sparse-checkout add src/<paquete>
# instala las dependencias de los paquetes agregados (mr_navigation y jc_driver se agregan más adelante)
rosdep install --from-paths ~/robots_ws/src --ignore-src -r -y --skip-keys "mr_navigation jc_driver"
colcon build --symlink-install --packages-select <paquete>
source install/setup.bash
```

Paso 7. Agregar los primeros paquetes: mr_base y mr_sim

Se agrega	Qué contiene
`src/mr_base`	Driver ROS 2 del RoboClaw, herramientas `check_roboclaw` y `mover`, regla udev en `udev/`.
`src/mr_sim`	Robot virtual: emulador del RoboClaw por un puerto serie virtual y lidar simulado.

Terminal – PC (en el robot se repite en la carpeta 11)

```
cd ~/robots_ws
git sparse-checkout add src/mr_base src/mr_sim
rosdep install --from-paths ~/robots_ws/src --ignore-src -r -y --skip-keys "mr_navigation jc_driver"
colcon build --symlink-install --packages-select mr_base mr_sim
source install/setup.bash
# explore la estructura del paquete
tree -L 2 src/mr_base
# mr_base, mr_sim
ros2 pkg list | grep mr_
# ahora sí: de dónde sale el paquete y cómo llegar a él (Paso 5)
ros2 pkg prefix mr_base
colcon_cd mr_base && pwd
```

Permisos y nombre fijo del puerto del RoboClaw (la regla udev crea `/dev/roboclaw` [7]; el detalle está en el DA-12.3):

Terminal – PC (en el robot se repite en la carpeta 11)

```
# cierre sesión y vuelva a entrar
sudo usermod -aG dialout $USER
sudo cp ~/robots_ws/src/mr_base/udev/99-mr-robot.rules /etc/udev/rules.d/
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Paso 8. Prueba sin robot: check_roboclaw con el RoboClaw virtual

Terminal – PC (1): RoboClaw virtual

```
ros2 run mr_sim roboclaw_emulator
```

Terminal – PC (2): diagnóstico

```
ros2 run mr_base check_roboclaw --port /tmp/roboclaw_emu --segundos 3
```

Resultado esperado:
`Version : USB Roboclaw 2x15a v4.2.8 (EMULADOR)`, la tensión de batería, el PID de velocidad de cada motor y el monitoreo de encoders durante 3 s. Detenga el emulador con Ctrl + C.

Con esto queda listo el **punto 5 del DA-10.1**: allí se repite el comando con el RoboClaw real conectado por USB (`--port /dev/roboclaw`).

Paso 9. En el computador del robot

El computador del robot recibe este mismo espacio de trabajo en la carpeta 11: los documentos de instalación de la Raspberry Pi, la Orin Nano y la AGX Orin terminan con el paso «Espacio de trabajo robots_ws en el robot y prueba del RoboClaw», que repite allí los Pasos 2 a 7 con sus comandos. A partir de ese momento, cada **Paso 0** de los documentos siguientes se hace en los dos equipos. En la Jetson Nano el procedimiento cambia (contenedor Docker y git 2.17, DA-11.2).

Paso 10. Recibir las correcciones del instructor

Cuando el instructor corrige o mejora un paquete, lo sube a GitHub. Cada equipo lo recibe así:

Terminal – PC y cada computador del robot (Jetson Nano: compile con run_nano.sh, DA-11.2)

```
cd ~/robots_ws
git pull
colcon build --symlink-install
source install/setup.bash
```

Si `git pull` avisa que sus cambios locales se sobrescribirían, guárdelos con `git stash`, ejecute `git pull` y recupérelos con `git stash pop`. Para ver qué cambió: `git log --oneline -5`.

5. VERIFICACIÓN Y EVIDENCIAS

Tabla 4. Lista de verificación.

#	Criterio	Evidencia
1	Ubuntu 22.04 y `ROS_DISTRO=humble`	Salida del Paso 1
2	`~/robots_ws` clonado y cargado en una terminal nueva	`git -C ~/robots_ws remote -v` y `printenv COLCON_PREFIX_PATH`
3	`mr_base` y `mr_sim` agregados y compilados	`git -C ~/robots_ws sparse-checkout list` y `ros2 pkg list | grep mr_`
4	`check_roboclaw` responde con el RoboClaw virtual	Salida del Paso 8

INSERTAR IMAGEN / CAPTURA (por el instructor):
Captura de `printenv COLCON_PREFIX_PATH`*,* `ros2 pkg prefix mr_base` *y la salida de* `check_roboclaw`*.*

6. SOLUCIÓN DE PROBLEMAS

Tabla 5. Problemas frecuentes.

Síntoma	Causa probable	Solución
`git clone` responde `Repository not found`	URL mal escrita	Copie la URL exacta del Paso 3
`git: 'sparse-checkout' is not a git command`	git anterior a 2.25 (Ubuntu 18.04)	En Ubuntu 22.04 no ocurre; en la Jetson Nano use el método del DA-11.2
`fatal: destination path '…/robots_ws' already exists`	Ya estaba clonado (por ejemplo, desde el cobot)	No lo clone de nuevo: continúe con el Paso 4
`ros2: command not found`	Entorno de ROS no cargado	`source /opt/ros/humble/setup.bash`; revise `~/.bashrc`
`Package 'mr_base' not found`	No se cargó `~/robots_ws/install/setup.bash`	Abra una terminal nueva; compruebe con `printenv COLCON_PREFIX_PATH`
`colcon_cd: command not found`	Falta la línea de `colcon_cd` en `~/.bashrc`	Paso 4 y terminal nueva
`check_roboclaw` no responde (`/tmp/roboclaw_emu`)	El emulador no está corriendo	Inícielo en la terminal (1) del Paso 8

REFERENCIAS

[1] Open Robotics, *REP 2000 – ROS 2 Releases and Target Platforms*. [En línea]. Disponible: https://www.ros.org/reps/rep-2000.html

[2] Open Robotics, *ROS 2 Documentation: Humble Hawksbill*. [En línea]. Disponible: https://docs.ros.org/en/humble/

[3] Open Robotics, *Creating a workspace* (ROS 2 Humble: underlay y overlay). [En línea]. Disponible: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-A-Workspace/Creating-A-Workspace.html

[4] Open Robotics, *Using colcon to build packages* (ROS 2 Humble; incluye colcon_cd). [En línea]. Disponible: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Colcon-Tutorial.html

[5] Open Robotics, *Managing Dependencies with rosdep* (ROS 2 Humble). [En línea]. Disponible: https://docs.ros.org/en/humble/Tutorials/Intermediate/Rosdep.html

[6] J. R. Cháux Cedeño, *Repositorio del curso JetCobot / robot móvil* (documentos de apoyo, GitHub). [En línea]. Disponible: https://github.com/jchauxdev/JetCobot

[7] systemd project, *udev — Dynamic device management* (manual). [En línea]. Disponible: https://www.freedesktop.org/software/systemd/man/latest/udev.html

[8] J. R. Cháux Cedeño, *robots_ws – paquetes ROS 2 del curso* (repositorio en GitHub, licencia Apache-2.0), 2026. [En línea]. Disponible: https://github.com/jchauxdev/robots_ws

[9] Git, *git-sparse-checkout – Reduce your working tree to a subset of tracked files*. [En línea]. Disponible: https://git-scm.com/docs/git-sparse-checkout